



Microcontroladores PIC32

Ignacio Cerda Torres

Alonso Pérez Vergara

Agustín Torres Bobadilla

22 de noviembre de 2024

1. Resumen

La familia de microcontroladores PIC32 es la serie de chips de 32 bits producida por Microchip Technology, estos están basados en arquitectura MIPS32, velocidades de hasta 252MHz, memoria flash hasta 2MB y 252KB de SRAM.

Estos microcontroladores son ampliamente utilizados en diversas áreas, debido a su gran gama de aplicaciones embebidas, áreas como procesos industriales, automotrices, IoT, etc.

Microchip technology provee software para la programación y simulación de los PIC3, todo esto puede ser encontrado en la página web de Microchip Technology. También cuenta con opciones de placas de desarrollo variadas para distintas versiones del PIC32

El desarrollo del proyecto se realiza mediante el entorno MPLAB X IDE y el compilador XC32, donde se usará el lenguaje de programación C++. Para completar este desarrollo, se debe tomar en consideración la relación de cada periférico con su puerto y registro respectivo, consultando las documentaciones correspondientes a los puertos y registros. Dentro del desarrollo de la implementación, se debe habilitar los puertos deseados para luego escribir las funciones y rutinas que serán exportadas al microcontrolador.

En definitiva, las aplicaciones del PIC32 son increíblemente variadas y útiles, siendo posible crear programas para que este realice virtualmente cualquier tarea.

2. Introducción

El PIC32 es una familia de microcontroladores reconocida por su amplia utilización. La familia posee una variedad de microcontroladores orientados

para diversos usos, de distinta cantidad de pines y consumo de corriente, memorias flash y RAM, y velocidad de ejecución. Algunos tienen módulos especializados para tareas que requieran cómputos especializados, como lo pueden ser una unidad de punto flotante o una unidad especializada de DSP. Los PIC32 —desde que fueron lanzados al mercado— tienen típicamente un *core* de arquitectura MIPS32. Sin embargo, en los últimos años Microchip —fabricante de los PIC32— ha sacado al mercado microcontroladores de arquitectura ARM bajo la marca PIC32.

El presente informe se enfocará principalmente en el PIC32 de arquitectura MIPS32, utilizando como referencia la *datasheet* del PIC32MZ2048EFM144.

3. PIC32

Estos microcontroladores PIC32 son usados principalmente en procesamiento de señales, control de motores y ampliamente en aplicaciones que estén compuestas de actividades internas que dependen completamente del proceso principal y se destacan por sus siguientes características internas y externas:

Arquitectura MIPS32: El núcleo del PIC32 está basado en la arquitectura MIPS32, un núcleo RISC (Reduced Instruction Set Computing) de 32 bits. El núcleo MIPS32 utiliza una arquitectura de pipeline de 5 etapas. Esto permite que múltiples instrucciones se procesen simultáneamente en diferentes etapas, optimizando el uso del procesador y aumentando su eficiencia.

Memoria: Ofrece grandes capacidades de RAM y Flash (hasta 2 MB de Flash y 512 KB de RAM). Logrando mayor almacenamiento de datos temporales y permanentes para el uso del programa.

Periféricos avanzados: Incluye controladores para USB, Ethernet, UART, PWM, entre otros. Ampliando las opciones de utilización del PIC.

RISC: Una de las ventajas de las arquitecturas RISC es que muchas de sus instrucciones se pueden ejecutar en un solo ciclo de reloj, lo que aumenta la velocidad del procesador. Este enfoque de simplificación facilita que el hardware sea más rápido y eficiente, permitiendo que más datos se almacenen temporalmente en el procesador sin necesidad de acceder a la memoria.

El modo de desarrollo del PIC32 para realizar una cierta tarea utilizando el software proporcionado por el fabricante MPLAB X IDE, considera en primer lugar en decidir que puertos del PIC32 nos son útiles y a través de que pin es posible acceder a ellos. Utilizando la hoja de datos del fabricante y documentación del compilador XC32 se deben notar los registros y bits en los que se debe escribir/leer en el código escrito en C/C++.

Una vez escrito el código, este debe ser probado utilizando un chip programador proporcionado por el fabricante, o bien, utilizando una placa de desarrollo que incluya un PIC32.

3.1. PIC32MZ2048EFM144

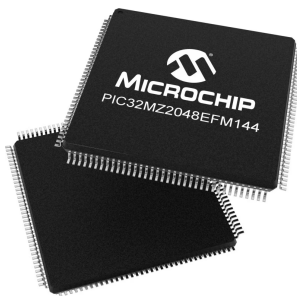


Figura 1: PIC32MZ2048EFM144 [2]

El PIC32MZ2048EFM144 es un microcontrolador perteneciente a la familia PIC32MZ EF, cuenta con velocidades de hasta 252MHz, 2MB de memoria flash, 512KB de SRAM (Static Random Access Memory), un oscilador interno para altas frecuencias de 8MHz y para bajas de 32kHz.

Este es principalmente implementado en aplicaciones industriales, automotrices, IoT, etc. Incluye un set variado de periféricos, tales como: 8-/16-

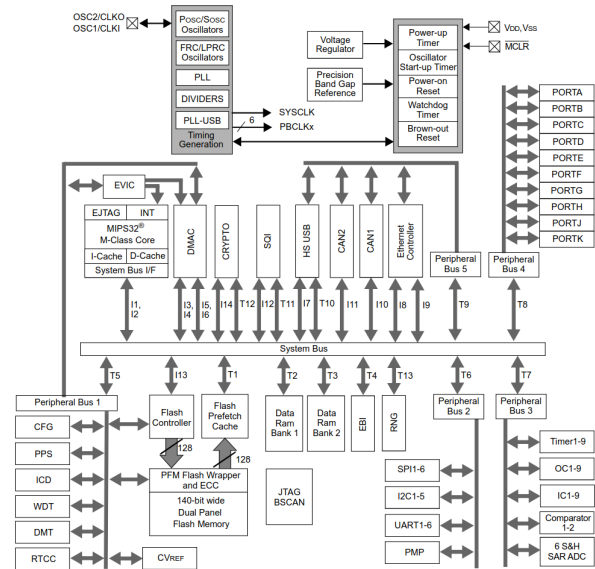


Figura 2: Diagrama de bloques de la familia PIC32 MZ EF [2]

bit Parallel Master Port (PMP) y interfaz Bus externa (EBI); 48-channel, 12-bit Analog-to-Digital Converter (ADC); Hi-Speed USB 2.0-compliant Device/Host/OTG; 10/100 Mbps Ethernet MAC; etc.

El 144 en el nombre del microcontrolador indica la cantidad de pines con los que cuenta, los cuales son manipulados a través de los registros con los que cuenta el microcontrolador. Estos registros se encuentran conectados al *peripheral Bus 4* que se conecta con la *Bus matrix*, mediante la manipulación de estos registros es posible habilitar puertos como salidas o entradas, leer los valores de estos o inhabilitarlos.

Un periférico importante dentro del PIC32 es su oscilador interno, El cual se describe de la siguiente forma según la Datasheet del mismo microcontrolador [2].

Por un lado se pueden destacar dos osciladores internos, el FRC Oscillator y el LPRC Oscillator. Siendo el FRC, un oscilador interno de alta frecuencia que incluye un oscilador RC que genera una frecuencia de referencia interna(8MHz), el microcontrolador puede usar esta frecuencia o algún prescalador que este dividido por N, según lo que el usuario desee.

El LPRC, es el otro oscilador interno que opera a bajas frecuencias, siendo más específicos a una

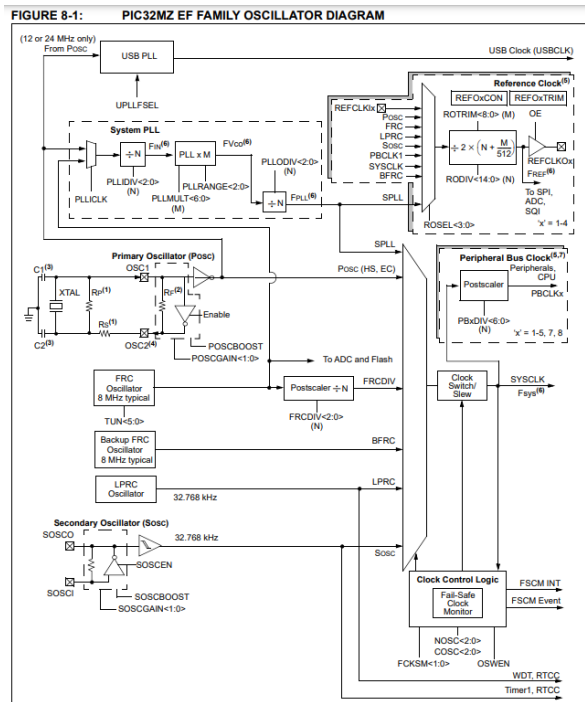


Figura 3: Diagrama del oscilador
[2]

frecuencia de 32.768[kHz] aproximadamente, utilizándose en tareas de de bajo consumo con sus pulsos de reloj, como puede ser indicar ciertos temporizadores de encendidos(RTCC, que indica el tiempo al cual se está operando, ya sea horas minutos o segundos dependiendo del operador) o también en modo watchdog que provoca un reset en el sistema para evitar que la unidad interna esté en un modo indeterminado.

Además se compone de dos conexiones externas(OSC1 y OSC2) que se conectan al *Primary Oscillator(Posc)* y al *Secondary Oscillator(Sosc)* respectivamente 3. En las conexiones OSC1 y OSC2 se puede conectar un cristal de cuarzo o incluso un circuito RC para obtener cierta precisión o valor para la frecuencia de operación del sistema, ya que estos osciladores externos generan una señal de reloj y pueden llegar a estabilizar dicha frecuencia. Este mismo caso se repite para el *Secondary Oscillator(Sosc)* si se requiere cierta precisión en su frecuencia.

También es importante mencionar el periférico ADC (*Analog To Digital Converter*), el cual está conectado al *Peripheral Bus 3* y cumple con la fun-

ción de digitalizar señales análogas. En el caso del PIC32MZ2048EFM144, el ADC cuenta con 12 bits, esto quiere decir que el voltaje de referencia es dividido en 2^{12} niveles, teniendo estos un intervalo de $\frac{V_{ref}}{2^{12}}$. El conversor análogo digital es ampliamente utilizado en procesamiento de señales para audio, comunicaciones, instrumentos de medición, etc.

4. Ejemplo básico de aplicación

4.1. Curiosity PIC32MZ EF 2.0 Development Board

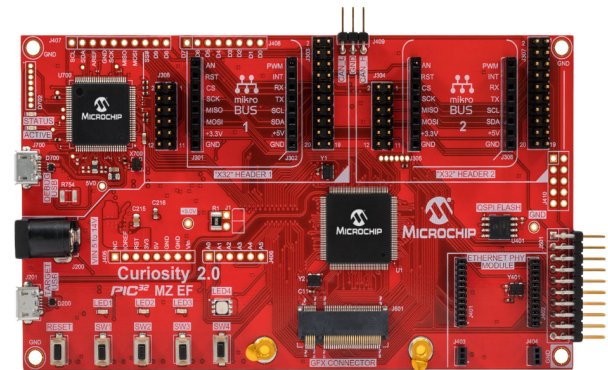


Figura 4: Curiosity PIC32MZ EF 2.0 Development Board [1]

La Curiosity PIC32MZ EF 2.0 Development Board es una placa que permite desarrollar fácilmente aplicaciones ya que el PIC32 incorporado tiene sus conexiones listas para programar el dispositivo e interactuar con el exterior mediante distintos puertos que la placa dispone, ya que de por si solo el PIC32 no se puede programar sin una placa como esta.

La programación y debugueo del PIC32 incluido se realiza mediante el chip PICKit On-Board 4(PKoB4) incluido en la placa (a la esquina superior izquierda). Este chip tiene una interfaz USB que se conecta al computador mediante el puerto micro-usb rotulado como "DEBUG USB". Este chip se conecta al PIC32 mediante en diversos puertos para programarlo y obtener valores de registros y memoria durante el debugueo, todo esto utilizando el software MPLAB X IDE junto con su compilador MPLAB XC Compiler.

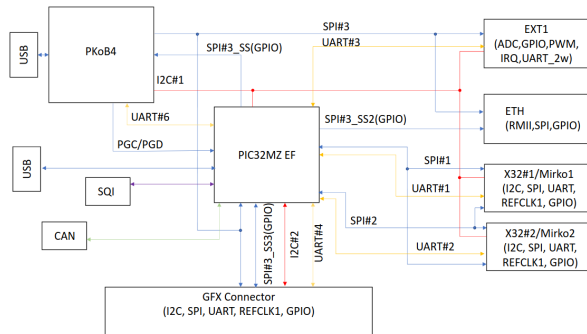


Figura 5: Diagrama de bloques básico de la placa de desarrollo [1]

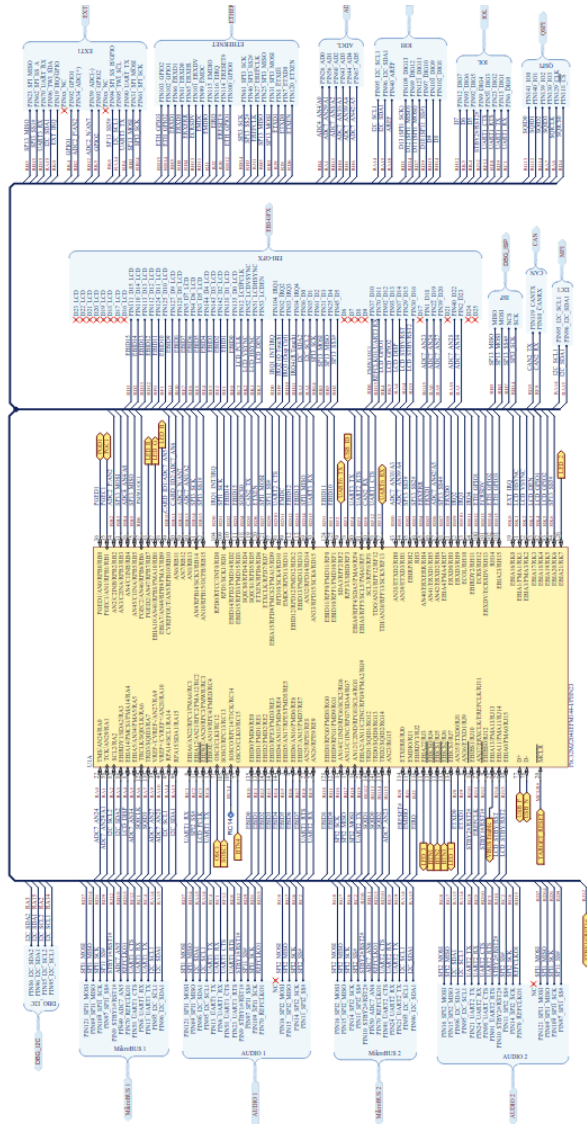


Figura 6: Conexiones de la placa en detalle [1]

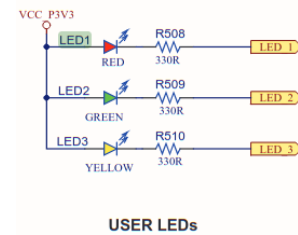


Figura 7: Conexiones de los LEDs [1]

Esta placa se usará para el ejemplo básico de aplicación.

4.2. Ruleta usando los leds

Para esta implementación se reproducirá el comportamiento de una ruleta utilizando los LED's y switches disponibles en la placa CURIOSITY 2.0.

Para el código se seguirá la siguiente estructura para la función main:

```
1 int main ( void )
2 {
3     setup();
4     while (1) {
5         task();
6     }
7
8     return ( EXIT_FAILURE );
9 }
```

La función setup realizará todas las acciones necesarias para habilitar (y configurar si es necesario) los puertos que se vayan a utilizar antes operar con ellos.

La función task por otro lado, hará la tarea que deseamos realizar, es decir, aquí vendrá el código del juego de la ruleta.

Primero es necesario definir los puertos correspondientes, para esto se consulta la data sheet correspondiente a la placa de desarrollo ([1]).

Una vez determinados los puertos a utilizar, se necesita conocer los registros mediante los que estos pueden ser manipulados, para esto se recurre a la Datasheet del microcontrolador ([2]).

Con los puertos y registros necesarios ya definidos, es posible proceder con la generación de código para programar el PIC32. Los registros y bits indicados por la Datasheet correspondientes a los LEDs 1, 2 y 3, son, PORTJ, PORTK y PORTJ, en los

Function	Description	Type	Port
SW1	Users switch	Input	RJ4
SW2	Users switch	Input	RJ5
SW3	Users switch	Input	RJ6
SW4	Users switch	Input	RC15
RESET	Hard reset of the PIC32	Input	-
LED1	User programmable LED. Single color	GPIO Output	RJ7
LED2	User programmable LED. Single color	GPIO Output	RK7
LED3	User programmable LED. Single color	GPIO Output	RJ3
LED4 (Red)	RGB LED Red channel	GPIO or PWM	RPB7 or OC8
LED4 (Green)	RGB LED Green Channel	GPIO or PWM	RPB8 or OC5
LED4 (Blue)	REB LED Blue Channel	GPIO or PWM	RPB9 or OC3

Figura 8: Puertos para LED's y Switches

Register	Bit Range	Bit 31	Bit 30	Bit 29	Bit 28	Bit 27	Bit 26	Bit 25	Bit 24	Bit 23	Bit 22	Bit 21	Bit 20	Bit 19	Bit 18	Bit 17	Bit 16	Bit 15	Bit 14	Bit 13	Bit 12	Bit 11	Bit 10	Bit 9	Bit 8	Bit 7	Bit 6	Bit 5	Bit 4	Bit 3	Bit 2	Bit 1	Bit 0
ANSELJ	31:16	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---
TRISJ	31:16	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---
PORTJ	31:16	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---
LATJ	31:16	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---

Figura 9: Registros de LED'S y Switches

bits 7, 7 y 3 respectivamente. La *Datasheet* sin embargo indica que estos sirven para leer lo ocurrido en los pines. Si se quiere utilizarlos para controlar los LEDs es necesario escribir un 0 en los registros TRISJ, TRISK y TRISJ, respectivamente en las posiciones mencionadas anteriormente. Posteriormente se pueden encender haciendo lo mismo en los registros LATJ, LATK y LATJ. Para acceder a estos registros mediante sus nombres, es necesario incluir el *header xc.h*, en donde se definen los identificadores de los registros como direcciones que el CPU comprenda.

A continuación vemos el código que enciende los tres leds. En la función *setup* habilitamos los LEDs, y en la función *task* se encienden.

```

1  #include <xc.h>
2
3  void setup(void)
4  {
5      // Habilitamos el port RJ7 como salida (LED1)
6      TRISJ &= 0xFF7F; // 1111 1111 0111 1111
7
8      // Habilitamos el port RK7 como salida (LED2)
9      TRISK &= 0xFF7F;
10
11     // Habilitamos el port RJ3 como salida (LED3)
12     TRISJ &= 0xFF7F;
13 }
14
```

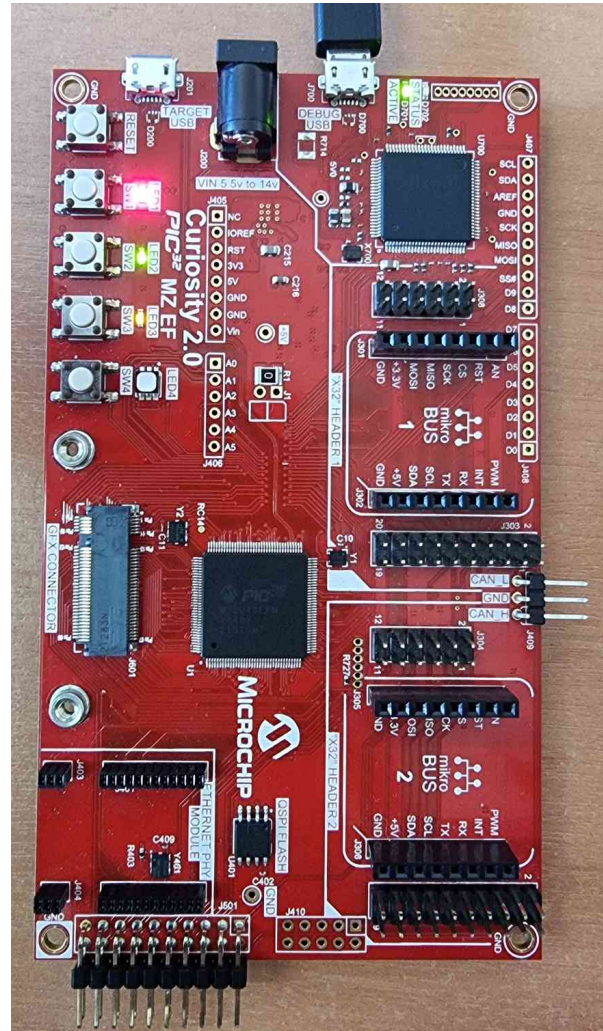


Figura 10: PIC32 con 3 LED's encendidos

```

void task(void)
{
    // Prendemos el LED1, LED2, LED3
    LATJ &= 0xFF7F;
    LATK &= 0xFF7F;
    LATJ &= 0xFF7F;
}

```

Como se puede observar en la figura 9 los tres LEDs efectivamente se encienden sin embargo las líneas de código que producen dicho resultado no son muy descriptivas; realizar cualquier tarea sería terriblemente engorrosa si se hiciera de este modo.

Por esta razón, es común en la programación de microcontroladores que se definan macros (que luego el preprocesador resolverá) abstrayendo la funcionalidad del código a algo más comprensible.

Se definen tres macros para la implementación de la ruleta. Uno para leer un bit de un cierto registro (`Get_bit(reg, bit)`), otro para poner un bit de un registro en 1 (`Set_bit(reg, bit)`) y un tercero para poner un bit de un cierto registro en 0 (`Clr_bit(reg, bit)`). La implementación de estas operaciones es sencilla utilizando los operadores lógicos y de desplazamiento que otorga C:

```
1 #define Get_bit(reg, bit) ((reg >> bit) & 0x1U)
2 #define Set_bit(reg, bit) (reg |= (0x1U << bit))
3 #define Clr_bit(reg, bit) (reg &= ~(0x1U << bit))
```

Con esto podemos definir fácilmente macros que habilitan encienden y apagan un cierto LED:

```
1 #define LED1_Enable() Clr_bit(TRISJ, 7)
2 #define LED1_ON() Clr_bit(LATJ, 7)
3 #define LED1_OFF() Set_bit(LATJ, 7)
4
5 #define LED2_Enable() Clr_bit(TRISK, 7)
6 #define LED2_ON() Clr_bit(LATK, 7)
7 #define LED2_OFF() Set_bit(LATK, 7)
8
9 #define LED3_Enable() Clr_bit(TRISJ, 3)
10 #define LED3_ON() Clr_bit(LATJ, 3)
11 #define LED3_OFF() Set_bit(LATJ, 3)
```

A partir de las hojas de datos determinamos que para operar con los switches 1, 2 y 3, hay que leer los bits 4, 5 y 6, del registro `PORTJ`. Además, a partir de la hoja de datos de la placa se observa que este puerto mostrará un valor 0 lógico cuando el switch sea presionado, y 1 cuando no.

Con dicha información se definen macros para los switches. Se lee el bit correspondiente y se determina si es igual a 0:

```
1 #define SW1_Get() (Get_bit(PORTJ, 4) == 0)
2 #define SW2_Get() (Get_bit(PORTJ, 5) == 0)
3 #define SW3_Get() (Get_bit(PORTJ, 6) == 0)
```

Así se obtiene una interfaz intuitiva para interactuar con los LEDs y switches. Sin embargo, si queremos que un LED se encienda por un cierto periodo y luego se apague, no podemos indicarle que se prenda y se apague simplemente, el cambio sería demasiado rápido para que se pueda apreciar.

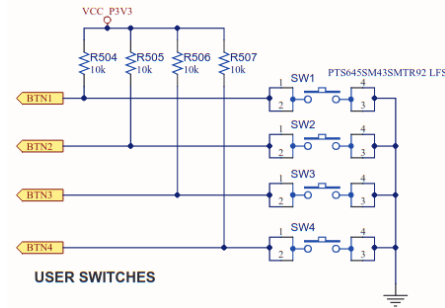


Figura 11: Conexiones de los switches

Register Number	Register Name	Function
0	Index	Index into the TLB array (MPU only).
1	Random	Randomly generated index into the TLB array (MPU only).
2	EntryLo0	Low-order portion of the TLB entry for even-numbered virtual pages (MPU only).
3	EntryLo1	Low-order portion of the TLB entry for odd-numbered virtual pages (MPU only).
4	Context/ UserLocal	Pointer to the page table entry in memory (MPU only). User information that can be written by privileged software and read via the RDHWR instruction.
5	PageMask/ PageGrain	PageMask controls the variable page sizes in TLB entries. PageGrain enables support of 1 KB pages in the TLB (MPU only).
6	Wired	Controls the number of fixed (i.e., wired) TLB entries (MPU only).
7	HWREna	Enables access via the RDHWR instruction to selected hardware registers in Non-privileged mode.
8	BadVAddr	Reports the address for the most recent address-related exception.
	BadInstr	Reports the instruction that caused the most recent exception.
	BadInstrP	Reports the branch instruction if a delay slot caused the most recent exception.
9	Count	Processor cycle count.
10	EntryHi	High-order portion of the TLB entry (MPU only).
11	Compare	Core timer interrupt control.

Figura 12: Registro Count

Es necesario una función que nos permita detenernos en cierta parte de la ejecución para reanudar pasado un periodo determinado de tiempo.

Utilizando `_CP0_SET_COUNT` y `_CP0_GET_COUNT` proporcionados por la `xc.h` para interactuar con el timer y poder tener una medida del tiempo. Estos macros “setean” y leen el valor del registro `count` dentro del PIC32, el cual corresponde a la cuenta de ciclos que ha completado el procesador desde que se comienza hasta que se mide.

Utilizando los procedimientos mencionados anteriormente, se define una función `delay` y macros que permitan detener la ejecución un tiempo deseado:

```
1 #define SYS_FREQ 200000000 // 200MHz
2 #define MS500 500000
3 #define MS250 250000
4 #define MS100 100000
5 #define MS50 50000
6
7 void delay(unsigned int us)
8 {
9     // Convertimos microsegundos en
10    // cuantos ticks del clock tomará
11    us *= SYS_FREQ / 1000000 / 2;
```

```

12
13 // Seteamos la cuenta del Core Timer a 0
14 _CP0_SET_COUNT(0);
15
16 // Esperamos que el Core timer alcance
17 // la cuenta calculada anteriormente
18 while (us > _CP0_GET_COUNT());
19 }

```

Una vez definido los macros para las LEDs, los switches y la función delay, se implementa el código base para lograr el funcionamiento de la ruleta usando LEDs.

El juego no comenzará hasta que se presione alguno de los 3 switches. Una vez presionado algún switch comienza una serie de encendido y apagado de los 3 LEDs en 2 instancias, con cierto retraso entre cada encendido y apagado. Luego comienza la ruleta a funcionar, comenzando siempre por el LED1 encendido para luego pasar al LED2 y finalmente al LED3, repitiendo el mismo ciclo en el mismo sentido (LED1, LED2, LED3). Cabe destacar que solo estará prendida una LED a la vez a medida que se encienden y se apagan con cierto *delay*, aumentando ese *delay* entre cada cambio de LED para darle una sensación de que esta desacelerando la ruleta.

Finalmente se entrega la LED ganadora al final del ciclo cuando se cumplen las vueltas máximas, comparando la selección de switch inicial con el LED ganador. Si el LED corresponde al switch, por ejemplo el LED ganador es el LED1 y se seleccionó el SW1 en un inicio, se gana el juego, en cualquier otro caso se perdería.

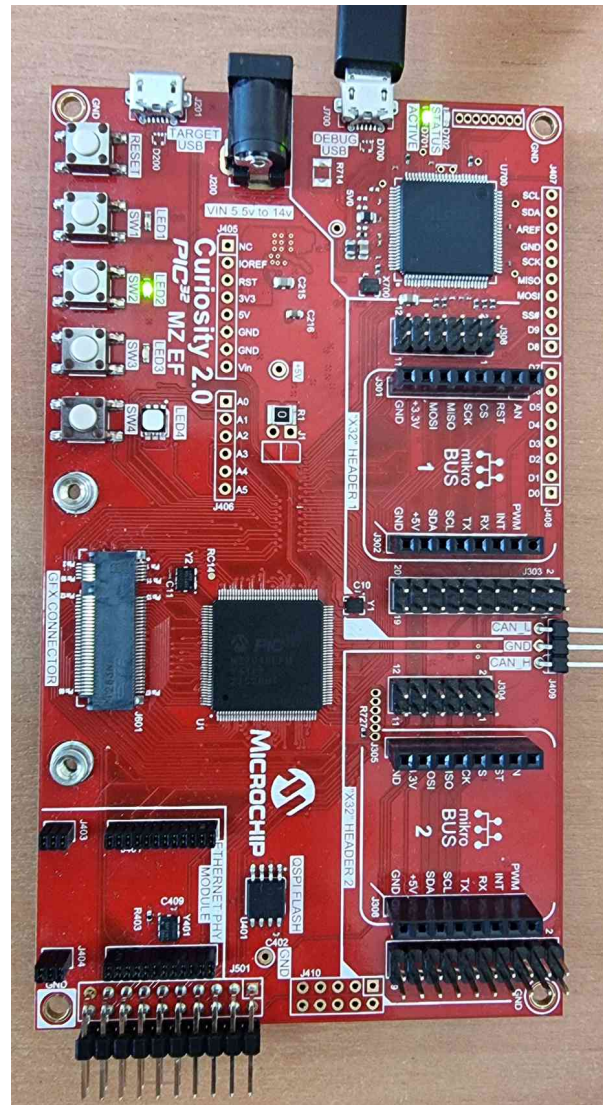


Figura 13: Ruleta

Referencias

- [1] *Curiosity PIC32MZ EF 2.0 Development Board Users Guide*. Microchip Technology. 2019.
- [2] *PIC32MZ Embedded Connectivity with Floating Point Unit (EF) Family*. Rev. H. Microchip Technology. 2021.